

Un Service Registry Decentralizzato basato su Gossip Anti-Entropy

Numo Sortino

Sistemi Distribuiti e Cloud Computing, A.A. 2025/26

ABSTRACT

Presentiamo il design e l'implementazione in Go di un service registry decentralizzato e tollerante ai guasti, basato su un overlay non strutturato e un protocollo di gossip anti-entropy push-pull su gRPC. La consistenza tra le repliche di ogni nodo è garantita da un versioning last-write-wins ispirato agli orologi scalari di Lamport. Il sistema affronta esplicitamente tre problemi emersi durante lo sviluppo (frammentazione del cluster in isole dopo un collasso correlato, mancata guarigione di una partizione di rete, contatti ridondanti infra-round) con tre possibili mitigazioni. Testiamo la soluzione proposta con scenari di test per l'aggiunta e la rimozione dinamica dei servizi, e per la simulazione di crash dei nodi al fine di verificarne la resilienza e la consistenza dei dati, e discutiamo le alternative di design scartate e i limiti noti del sistema.

1 Introduzione e requisiti

Il progetto implementa un registro di servizi che permette a servizi arbitrari di registrare e scoprire i propri endpoint senza affidarsi a un server centrale. I requisiti (decentralizzazione, tolleranza ai guasti con consistenza, scalabilità rispetto ad aggiunte/rimozioni dinamiche) guidano ogni scelta discussa nella sezione 2: l'assenza di un coordinatore centrale esclude a priori soluzioni basate su consenso forte (Raft/Paxos), e orienta verso un modello peer-to-peer.

2 Architettura e scelte progettuali

2.1 Overlay: nodi completamente connessi vs. DHT

Nodi completamente connessi, con piena replica (scelta adottata). Ogni nodo conserva una copia completa dell'intero registro, quindi può rispondere da solo a qualsiasi richiesta di lookup, senza doverla inoltrare ad altri nodi e senza bisogno di alcuna struttura di instradamento da mantenere coerente quando un nodo entra o esce dal cluster. Lo svantaggio è che lo stato conservato da ciascun nodo cresce linearmente col numero di nodi N (invece che in modo logaritmico, come in una DHT), e il traffico generato dal gossip aumenta di conseguenza al crescere di N . La piena connessione è garantita fin dall'avvio, non scoperta gradualmente: il file di configurazione di ogni nodo elenca già gli indirizzi di tutti gli altri $N-1$ nodi del cluster (generati insieme dallo stesso script), così il

primo round di gossip può già raggiungere chiunque.

DHT, Chord/Kademlia (scartata). In una DHT ogni nodo conosce solo una piccola parte del cluster, e una ricerca viene risolta passando attraverso pochi altri nodi invece che localmente: un compromesso che permette di scalare a migliaia di nodi. Il prezzo è una struttura di instradamento (la finger table di Chord, o l'equivalente in Kademlia) da tenere coerente a ogni ingresso o uscita di un nodo, oltre a uno schema di hashing consistente per distribuire le chiavi: più complessità implementativa e più punti in cui può nascondersi un bug, per un vantaggio di scalabilità che alla dimensione target di questo progetto (decine di nodi) non serve davvero.

Con un numero di nodi dell'ordine delle decine (i test coprono 10–40 nodi, sezione 5), il costo $O(N)$ della piena replica resta trascurabile, mentre il beneficio (ogni nodo risponde a qualunque client senza bisogno di redirectione o instradamento) è concreto a questa scala.

2.2 Propagazione: gossip anti-entropy

Gossip anti-entropy push-pull (scelta adottata). Quando un client registra o rimuove un servizio, l'operazione viene applicata subito, solo sul nodo contattato: il client non aspetta che l'informazione raggiunga gli altri nodi, quindi la scrittura resta veloce indipendentemente da quanto sia grande il cluster. La propagazione avviene poi in un secondo

momento, tramite gli scambi periodici di gossip fra coppie di nodi: se un messaggio va perso o un nodo è temporaneamente irraggiungibile non serve nessun meccanismo di conferma o ritrasmissione dedicato, perché al giro di gossip successivo l'informazione verrà comunque ripresa e ritrasmessa da un altro nodo. Lo svantaggio è che, subito dopo una scrittura, gli altri nodi possono ancora avere una versione non aggiornata: esiste una finestra di tempo, misurata nel Test 2, in cui una lettura su un nodo diverso da quello di scrittura può restituire un dato non ancora propagato.

Flood-on-write (scartata). Ogni scrittura dovrebbe raggiungere sincronamente tutti i nodi raggiungibili, aumentando la latenza percepita dal client. Inoltre un nodo irraggiungibile proprio al momento della scrittura perderebbe l'aggiornamento per sempre, a meno di aggiungere un meccanismo separato che lo faccia recuperare in un secondo momento.

Retry lato client su più nodi (scartata). Sposta la responsabilità di conoscere la topologia del cluster sul client, contraddicendo l'obiettivo di decentralizzazione trasparente (“un client contatta un nodo qualsiasi”).

Adottiamo push-pull: ogni scambio invia lo stato locale e riceve quello del peer in un'unica chiamata RPC, per massimizzare l'informazione trasferita per round.

2.3 Due protocolli, due pubblici

gRPC (nodo↔nodo). Canale interno, alta frequenza (un round al secondo per nodo), schema tipizzato generato dal `.proto`, framing binario Protocol Buffers su HTTP/2: adatto a un contratto stabile tra parti che eseguono lo stesso codice.

HTTP+JSON (client→nodo). Canale esterno: deve restare raggiungibile da un client scritto in qualunque linguaggio, con `curl` o un browser, senza generare stub. La stessa filosofia protocol-agnostic guida il controllo di raggiungibilità dei servizi registrati (sezione 3.4): il registro non impone alcun protocollo applicativo né ai client dell'API né ai servizi che vi si registrano.

2.4 Consistenza: last-write-wins

Ogni voce del registro (nel codice, `ServiceEntry`) porta con sé un'informazione di versione: una coppia formata da un contatore che si incrementa a ogni modifica da parte del client e dall'identificativo del nodo che lo ha inserito per ultimo nel registro (nel codice, `Version{Counter, OwnerNodeID}`): lo

stesso principio dei clock scalari di Lamport, con un pareggio risolto confrontando gli identificativi dei nodi. Una “modifica” non è solo la prima registrazione: un client può ri-registrare più volte lo stesso servizio, ad esempio dopo aver cambiato porta o endpoint, e ogni ri-registrazione incrementa di nuovo il contatore, aggiornando anche il nodo indicato come ultimo scrittore. Quando due nodi si scambiano due versioni della stessa voce, vince quella col contatore più alto; a parità di contatore, vince l'identificativo del nodo lessicograficamente maggiore. Il nodo con il contatore più basso non tiene la propria versione: si riallinea a quella vincente, ereditandone contatore e identificativo del nodo, così che entrambi i nodi restino sincronizzati sulla stessa versione dopo lo scambio. La rimozione di un servizio non cancella subito la voce corrispondente, ma la marca come rimossa (stato `REMOVED`, una sorta di lapide versionata): una cancellazione fisica immediata farebbe sì che un nodo ancora in possesso della vecchia versione la resusciti al successivo scambio.

3 Implementazione

3.1 Selezione del peer: least-recently-contacted

Ogni round seleziona il peer *least-recently-contacted*, cioè quello non contattato da più tempo, così nessun peer resta trascurato a lungo, a differenza di quanto può accadere con la selezione uniforme casuale.

3.2 Ricalcolo infra-round

Ogni nodo, a ogni round, contatta `peers_per_round` nodi (parametro di configurazione). Il punto chiave è *quando* vengono decisi: non tutti insieme, in un'unica lista calcolata prima di iniziare i contatti del round, ma uno alla volta, richiamando la selezione del peer (sezione 3.1) *dentro* il ciclo, subito prima di ogni singolo contatto. Nel codice troviamo infatti (semplificato):

```
contactPeer(peers[0])
for i := 1; i < peersPerRound; i++ {
    peers := selectLeastContacted(1) // richiamata a ogni iterazione
    contactPeer(peers[0])
}
```

La selezione, cioè la chiamata a `selectLeastContacted`, sta dentro il corpo del ciclo, non prima: a ogni iterazione riparte dallo stato più aggiornato disponibile in quel momento, invece che da un elenco deciso una volta sola a inizio round.

Il motivo per cui questo conta riguarda il modo in cui il tempo dall'ultimo contatto di un peer può aggiornarsi. Non succede solo quando questo nodo

contatta direttamente quel peer: può succedere anche indirettamente, quando un *altro* nodo qualsiasi, durante il normale scambio di gossip, riporta a questo nodo una dichiarazione più recente su quel peer, per esempio perché quell'altro nodo lo ha contattato lui stesso poco prima. Nel codice troviamo infatti:

```
if incoming.State == Alive {  
    incoming.LastContactAt = time.Now()  
}
```

Quando arriva una dichiarazione su un peer che lo dice vivo, il tempo dall'ultimo contatto di quel peer viene aggiornato ad ora, anche se non è stato questo nodo a contattarlo: gli è bastato venirlo a sapere da un terzo nodo nello stesso scambio di gossip.

Questo è un contatto indiretto. Ne segue che, se il primo scambio del round di questo nodo porta con sé anche una notizia fresca su un peer che sarebbe stato scelto pure per il secondo contatto del round, quel peer non è più il meno recentemente contattato: un elenco di contatti deciso tutto insieme a inizio round non se ne accorgerebbe, e lo ricontatterebbe comunque inutilmente, sprecando quel contatto. Ricalcolando la selezione a ogni iterazione, invece, questo spreco viene evitato.

3.3 Guasti dei nodi: due possibili mitigazioni

Il meccanismo che rileva i nodi guasti tiene traccia, per ogni peer, del tempo trascorso dall'ultimo contatto riuscito, e lo fa passare attraverso tre stati (vivo, sospetto, morto; nel codice,

ALIVE→SUSPECTED→DEAD), al superamento di due soglie di tempo crescenti. Un peer marcato come morto viene escluso dalla selezione normale dei peer da contattare: è una scelta corretta, perché un nodo non ha motivo di riprovare chi ha già dichiarato morto. Questa scelta, però, porta con sé due problemi distinti, emersi osservando il comportamento del cluster containerizzato durante lo sviluppo, e affrontati con due meccanismi indipendenti.

Il primo problema riguarda un collasso correlato: cosa succede se, non uno solo, ma *tutti* i nodi conosciuti da un peer risultano morti insieme? Può capitare ad esempio se l'intera macchina virtuale viene sospesa per un po' e, al suo risveglio, tutti i timeout scattano in blocco. In questo caso il nodo non avrebbe più alcun candidato vivo da contattare, il che è un problema: il cluster resterebbe silenzioso pur essendo i processi ancora vivi, semplicemente perché nessuno prova più a ricontattare nessuno. La soluzione: quando

l'insieme dei peer vivi o sospetti è vuoto, la funzione di selezione dei peer (nel codice, `SelectLeastContacted`) non si limita più a un solo contatto per round, ma ripescava *tutti* i peer morti in un colpo solo, ignorando il limite configurato di contatti per round (`peers_per_round`).

Il secondo problema riguarda invece una partizione di rete, ed è più subdolo perché il meccanismo appena descritto non lo copre: quella soluzione scatta solo quando l'insieme dei peer vivi è completamente vuoto, ma durante una partizione ogni lato della rete mantiene comunque dei peer vivi al proprio interno: quelli dallo stesso lato. Il ripiego a contatto multiplo, quindi, non scatta mai, e i nodi dell'altro lato restano marcati come morti anche dopo che la rete torna disponibile, perché nessuno prova più a ricontattarli. Per questo, indipendentemente da quanti peer vivi esistano, ogni tot round (un intervallo configurabile, nel codice `dead_check_every_n_rounds`) viene comunque contattato il peer morto contattato meno di recente. Basta un solo contatto riuscito, da un solo nodo qualsiasi tra i due lati, perché la notizia si propaghi a tutto il cluster attraverso il normale scambio di gossip.

3.4 Salute dei servizi registrati

Il registro non aveva inizialmente alcun modo di distinguere un servizio morto per sempre da uno momentaneamente irraggiungibile: una volta registrato, restava attivo (stato `ACTIVE`) a tempo indeterminato. Un ciclo periodico separato, indipendente dal gossip (nel codice, `doHealthCheck`), apre e chiude una connessione TCP verso l'endpoint di ogni servizio noto, apposta per non aggiungere latenza di rete al percorso di lettura più frequente del sistema; in base all'esito aggiorna lo stato del servizio ad attivo o sospetto (`ACTIVE/SUSPECTED`) con una nuova versione, con la stessa tecnica di Lamport già usata per le voci del registro. Il probe è deliberatamente solo un *TCP connect*, non un vero health-check applicativo: è l'unico controllo protocol-agnostic possibile, dato che il registro non sa nulla del protocollo applicativo dietro un endpoint arbitrario (coerente con la scelta HTTP+JSON della sezione 2.3). È un segnale volutamente debole (discusso in sezione 6): un processo con la porta aperta ma bloccato risulterebbe comunque `ACTIVE`.

3.5 Sfasamento degli intervalli di gossip

Nodi diversi che contattano lo stesso peer nello stesso istante sono una fonte di ridondanza osservata empiricamente. L'intervallo di tempo tra un giro di gossip e il successivo, per quel nodo (nel codice, `round_interval_ms`), è quindi sfasato nodo per nodo secondo la formula:

```
offset(N) = (10 × (N-1)) mod 200
round_interval_ms(N) = 1000 + offset(N)
```

Il modulo, non un incremento lineare puro, mantiene lo spread limitato a 200 ms indipendentemente dal numero di nodi: con un incremento lineare, un cluster di centinaia di nodi porterebbe l'ultimo ad avere un intervallo quasi

triplo rispetto al primo: non più un lieve sfasamento ma un ritmo di gossip effettivamente diverso da nodo a nodo.

4 Piattaforma e strumenti

Go 1.25 (goroutine per i cicli di gossip/sweep/health-check, indipendenti tra loro); gRPC e Protocol Buffers per il canale nodo-nodo; libreria standard `net/http` per l'API client-facing; `gopkg.in/yaml.v3` per la configurazione esterna. Containerizzazione con Docker e orchestrazione con Docker Compose, con generatori parametrici per cluster di dimensione variabile. Deploy su istanza Amazon EC2 via AWS Academy Learner Lab.

5 Risultati sperimentali

Due scenari sono stati eseguiti sul cluster containerizzato, interamente sull'istanza Amazon EC2 richiesta dalle specifiche, a scale diverse (10 o 20 nodi a seconda dell'esperimento). Il Test 2 riceve l'attenzione principale: anziché un crash a tempi fissi arbitrari, ogni nodo del cluster viene fatto crashare a una frazione (1/12, 1/10, 1/8, 1/4) del proprio `round_interval_ms` individuale (sezione 3.5), per un totale di 80 prove (20 nodi × 4 frazioni), così da distinguere quantitativamente il contributo del *push* (il round di gossip uscente del nodo stesso, prima di morire) da quello del *pull* (un contatto indipendente da un altro nodo) alla sopravvivenza del dato appena scritto, restringendo l'osservazione alla finestra temporale in cui il rischio di perdita è concreto. Le mitigazioni architetturali descritte in sezione 3.3 restano tutte parte del design anche dove non hanno un test

dedicato in questa sezione; i CSV completi di ogni esperimento accompagnano questa relazione. **Test 1: registrazione/rimozione dinamica, con fotografia dell'evoluzione** Su un cluster isolato a 10 nodi, 16 scansioni (`List()` su tutti i nodi) catturano l'evoluzione del registro. A differenza di una prima versione dell'esperimento, qui ogni rimozione o aggiunta è eseguita su un *singolo* nodo per volta, con una scansione immediatamente successiva: così si vede il nodo che ha scritto localmente, da solo, prima che il gossip raggiunga gli altri. Sequenza: registrazione di 8 servizi; rimozione di S1 (solo su node-1); rimozione di S2 (solo su node-2); aggiunta di S9 (solo su node-9); aggiunta di S10 (solo su node-10); infine, a chiusura del test, rimozione contestuale di S3, S4, S5. Ogni cella riporta il numero di servizi attivi noti a quel nodo in quell'istante.

Tabella 1a. Test 1, scansioni 1-8: costruzione del registro, poi rimozione di S1 e di S2, un nodo alla volta.

Nodo	T+2.3s	T+3.3s	T+3.5s	T+4.0s	T+4.7s	T+4.8s	T+5.4s	T+6.0s
node-1	8	8	7	7	7	7	7	6
node-2	7	8	8	8	7	6	6	6
node-3	8	8	8	7	7	7	7	6
node-4	8	8	8	7	7	7	6	6
node-5	5	8	8	7	7	7	6	6
node-6	8	8	8	7	7	7	6	6
node-7	8	8	8	8	7	7	7	6
node-8	8	8	8	7	7	7	7	6
node-9	8	8	8	7	7	7	7	6
node-10	8	8	8	8	7	7	6	6

Tabella 1b. Test 1, scansioni 9–16: aggiunta di S9 e di S10, un nodo alla volta, poi rimozione finale di S3, S4, S5.

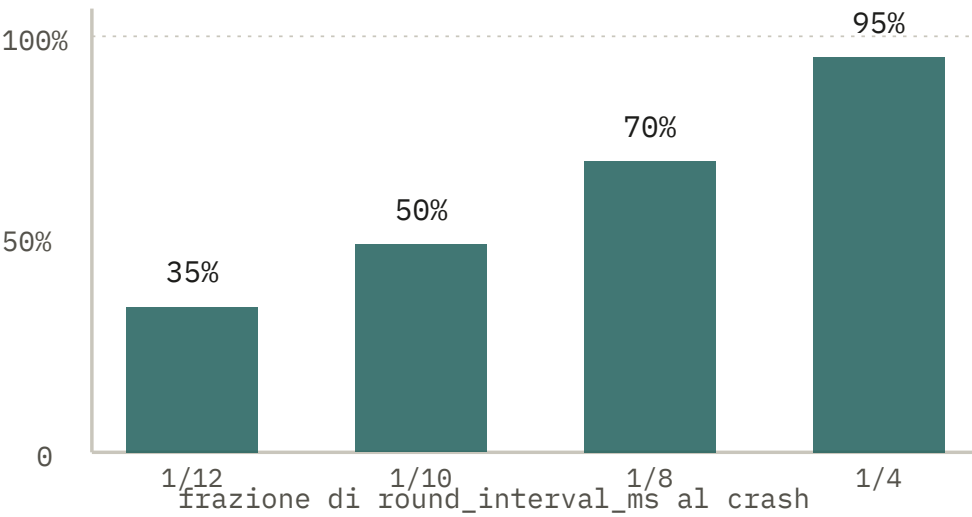
Nodo	T+6.1s	T+6.7s	T+7.3s	T+7.5s	T+8.0s	T+8.6s	T+9.0s	T+10.6s
node-1	6	6	6	6	8	8	8	5
node-2	6	6	6	7	8	8	8	5
node-3	6	6	6	7	8	8	7	5
node-4	6	6	7	7	7	8	6	5
node-5	6	6	7	7	8	8	7	5
node-6	6	6	7	7	8	8	8	5
node-7	6	6	6	6	8	8	6	5
node-8	6	6	7	7	8	8	7	5
node-9	7	7	7	7	8	8	8	5
node-10	6	6	7	8	8	8	7	5

Le quattro celle evidenziate in verde mostrano la scrittura visibile su un *solo* nodo, prima che il gossip la propaghi: a T+3.5s solo node-1 è sceso a 7 dopo la rimozione di S1 (gli altri nove sono ancora a 8); lo stesso a T+4.8s per S2 (solo node-2 a 6), a T+6.1s per S9 (solo node-9 a 7), a T+7.5s per S10 (solo node-10 a 8). In ogni caso, entro due o tre scansioni (meno di un secondo) tutti i 10 nodi si allineano, grazie al versioning last-write-wins di sezione 2.4.

La rimozione finale di S3, S4, S5 (T+9.0s), su tre nodi diversi in rapida sequenza, produce uno stato transitorio più frammentato prima di convergere a 5 su tutti (T+10.6s).

Test 2: crash e consistenza push/pull

Il grafico sotto riporta il tasso di sopravvivenza del servizio in funzione della frazione di tempo di round di gossip (nel codice, `round_interval_ms`) trascorsa prima del crash del nodo proprietario.



Test 2 (80 prove, 20 nodi): sopravvivenza del servizio in funzione della frazione di `round_interval_ms` trascorsa prima del crash: crescente da 35% (1/12 round) a 95% (1/4 round). Il dato più rilevante non è visibile in altezza: in nessuna delle quattro fasce si osserva un salvataggio dovuto al solo push (0 casi su tutte); ogni sopravvivenza rilevata è dovuta al pull (da solo, o insieme al push): a tempi così brevi il nodo morente quasi mai completa un proprio round di gossip uscente prima di morire.

6 Limitazioni riscontrate

- **Segnale di salute debole.** Il controllo di raggiungibilità è un TCP connect, non un health-check applicativo: un processo con la porta aperta ma bloccato risulta comunque `ACTIVE` (sezione 3.4).
- **Contatti in sequenza, non in parallelo.** Sia il ripiego da collasso globale sia il controllo anti-partizione contattano i peer uno alla volta (nel codice, la funzione `doRound`), non in parallelo tramite più goroutine concorrenti. Parallelizzare i contatti è un miglioramento naturale non implementato per contenimento dei tempi di progetto.
- **Spareggio simbolico, non temporale.** A parità di `Counter`, vince l'`OwnerNodeID` lessicograficamente maggiore: durante una partizione non esiste un orologio condiviso affidabile, quindi il vincitore non è necessariamente chi ha scritto per ultimo in senso cronologico reale, ma è garantito essere lo stesso ovunque nel cluster. Non verificato con un test dedicato in questa relazione (sezione 2.4 ne descrive il meccanismo).
- **Nessun coordinatore (leader).** Qualsiasi nodo accetta scritture in ogni momento, senza arbitraggio: è proprio questo ad aprire lo spareggio simbolico del punto precedente. Un leader eletto lo risolverebbe alla radice, in due

modi possibili: instradando tutte le scritture di una chiave attraverso un unico nodo che le serializza in un ordine totale (niente più scritture concorrenti sulla stessa chiave, per definizione); oppure, durante una partizione, facendo scrivere solo il lato col leader, lasciando l'altro indisponibile finché la rete non guarisce. È il trade-off CAP opposto: consistenza dell'ordine invece di piena disponibilità di scrittura.

- **Ridondanza cross-nodo solo mitigata.** Nodi diversi possono ancora, in casi rari, contattare lo stesso peer in una finestra ravvicinata; lo sfasamento degli intervalli (sezione 3.5) riduce ma non elimina il fenomeno, osservato empiricamente in fase di sviluppo.

7 Conclusioni

Il sistema soddisfa i requisiti di decentralizzazione, tolleranza ai guasti e scalabilità dinamica richiesti, con tre meccanismi aggiuntivi (il ripiego a contatto multiplo, il controllo anti-partizione periodico e il controllo di salute dei servizi) introdotti osservando il comportamento del cluster durante lo sviluppo. Le scelte architetturali privilegiano la semplicità (nodi completamente connessi, TCP-only health check, due protocolli separati per due pubblici distinti) rispetto a soluzioni più complesse ma non necessarie alla scala target, con i trade-off resi espliciti in ogni sezione.